Here's a **clear side-by-side comparison table** of your listed AI model types, highlighting **what they are, how they work, key differences, and real-life examples**:

AI Model Type	Core Concept	How It Works	Key Difference from Others	Strengths	Limitations	Real-Life Example / Use Case
Machine Learning – Linear Regression	Predicts a continuous value based on input features	Fits a straight line (y = mx + c) to model relationship between inputs & outputs	Simpler, interpretable, requires less data; unlike deep learning, no complex layers	Easy to interpret, fast, requires less computing power	Not good for non-linear patterns	Predicting **house prices** based on size, location, and amenities
Deep Learning – CNN (Image Classification)	Recognizes and classifies images by learning patterns in pixels	Uses convolutional filters to detect edges, shapes, and features in layers	Specialized for visual data; much more powerful than basic ML	High accuracy for visual tasks, learns complex patterns	Requires large datasets & high compute	Identifying **cats vs. dogs** in images for photo tagging
NLP – BERT for Sentiment Analysis	Understands and processes human language contextually	Transformer-based model learns word meaning from both left & right context	Focused on language understanding; bidirectional context unlike older NLP models	Captures deep meaning, handles ambiguity	Large model size, slow to train from scratch	Analyzing **customer reviews** to see if feedback is positive or negative
Generative Models – GAN	Creates new data similar to training examples	Two networks (Generator & Discriminator) compete to improve output realism	Generates new data rather than predicting or classifying	Can create realistic images, videos, voices	Can be unstable to train, risk of misuse (deepfakes)	Creating **artwork or synthetic human faces** that look real
Reinforcement Learning – Q-Learning	Learns to make decisions	Maintains a table of states and	Focuses on decision-making over	Works without labeled	Struggles with large	Teaching a **robot to navigate a**

	through trial & error to maximize reward	actions with rewards	time; no explicit training dataset	data, good for sequential decision problems	state spaces, slow convergence	**maze**
Computer Vision – YOLO (Object Detection)	Detects & locates multiple objects in an image in real-time	Divides image into grid, predicts bounding boxes & class probabilities	Detects *and* localizes objects, unlike classification-only CNNs	Very fast, real-time detection	May miss very small objects	Detecting **cars and pedestrians** for self-driving cars
Multi-Modal – CLIP	Understands both text & images together	Learns joint embeddings for text and images, enabling cross-modal search	Handles multiple data types at once	Can match descriptions to images and vice versa	Requires large-scale pretraining	Searching **images by text description** ("red sports car in snow")
Specialized – AlphaFold	Predicts 3D protein structures from amino acid sequences	Uses deep learning to map sequences to 3D shapes	Domain-specific, designed for biology	Major breakthrough in biology, accurate protein folding prediction	Only for protein-related tasks	Accelerating **drug discovery** by predicting protein structures

If you like, I can also create a **second table** showing **step-by-step how each model type works with a small conceptual Python example** so you see them in action. That would make it more practical for learning.

Nice — below I've broken each model down **step-by-step** and included a **small, conceptual Python snippet** you can read or adapt. The code is intentionally minimal so you can see the core idea quickly (real training needs more data, hyperparameter tuning, and compute).

1) Machine Learning — Linear Regression

Steps (workflow)

1. Collect dataset of feature(s) X and continuous target y.
2. Clean & engineer features (handle missing values, scale if needed).
3. Split into train / validation sets.
4. Fit a linear model to minimize squared error (learn coefficients).
5. Evaluate (MSE, R^2) and use model to predict new values.

Conceptual Python (scikit-learn)

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# toy data
X = np.array([[50], [60], [70], [80], [90]])  # e.g., house size (sq ft)
y = np.array([150, 180, 210, 240, 270])      # price (k$)

X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=0)
model = LinearRegression()
model.fit(X_train, y_train)
pred = model.predict(X_val)
print("Coef:", model.coef_, "Intercept:", model.intercept_)
print("MSE:", mean_squared_error(y_val, pred))
```

---

2) Deep Learning — CNN (Image Classification)

Steps (workflow)

1. Prepare labeled image dataset; resize & normalize images.
2. Optionally augment data (flip, crop, color jitter).
3. Build CNN: conv → activation → pooling layers → dense head.
4. Train with mini-batches using cross-entropy loss and optimizer (Adam/SGD).
5. Validate, tune, and deploy for inference.

Conceptual Python (Keras)

```
from tensorflow.keras import layers, models

model = models.Sequential([
   layers.Input(shape=(64,64,3)),
```

```
    layers.Conv2D(32, 3, activation='relu'),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, activation='relu'),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')  # 10 classes
])
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
# model.fit(train_images, train_labels, epochs=10, batch_size=32)
```

---

3) NLP — BERT for Sentiment Analysis

Steps (workflow)

1. Collect labeled text (e.g., reviews + sentiment labels).
2. Tokenize texts with BERT tokenizer (produces input_ids, attention_mask).
3. Load pre-trained BERT with a classification head.
4. Fine-tune on your dataset (small learning rate, few epochs).
5. Evaluate (accuracy, F1) and use softmax probabilities for inference.

Conceptual Python (Hugging Face Transformers)

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModelForSequenceClassification.from_pretrained("bert-base-uncased",
num_labels=2)

texts = ["I love this!", "Terrible experience."]
inputs = tokenizer(texts, padding=True, truncation=True, return_tensors="pt")
outputs = model(**inputs)  # logits: shape (batch, num_labels)
# probs = softmax(outputs.logits, dim=1)
```

---

4) Generative Models — GAN (Generative Adversarial Network)

Steps (workflow)

1. Prepare dataset of real examples (e.g., images).
2. Define two networks: **Generator** (creates fake samples) and **Discriminator** (classifies real vs fake).
3. Training loop:
    - Train discriminator on real samples (label=1) and generator's fakes (label=0).
    - Train generator to **fool** discriminator (update generator with discriminator's gradients).
4. Alternate updates until generator produces realistic outputs.
5. Sample generator for new synthetic data.

Conceptual Python (PyTorch-like pseudocode)

```
# generator(z) -> fake_img, discriminator(img) -> prob_real
# losses: BCE for both; alternate optimizer steps
for epoch in range(epochs):
   # 1) train discriminator on real and fake
   # 2) train generator to maximize discriminator(fake) (i.e., fool it)
   pass
```

(Real code defines NN modules, optimizers, and explicit loss/backprop steps.)

---

5) Reinforcement Learning — Q-Learning (tabular)

Steps (workflow)

1. Define environment with discrete states and actions and reward function.
2. Initialize Q-table Q[state, action] = 0.
3. For each episode: pick action using ε-greedy policy, observe reward and next_state.
4. Update Q using Bellman update:
   Q[s,a] += alpha * (r + gamma * max_a' Q[s',a'] - Q[s,a]).
5. Decay ε and repeat until convergence or acceptable policy learned.

Conceptual Python (tabular Q-learning)

```
import numpy as np
n_states, n_actions = 16, 4
Q = np.zeros((n_states, n_actions))
alpha, gamma, epsilon = 0.1, 0.99, 0.2

for episode in range(1000):
   state = env.reset()
   done = False
   while not done:
      if np.random.rand() < epsilon:
         action = np.random.randint(n_actions)
      else:
         action = np.argmax(Q[state])
      next_state, reward, done, _ = env.step(action)
      Q[state, action] += alpha * (reward + gamma * Q[next_state].max() - Q[state, action])
      state = next_state
```

(env can be a simple gridworld or gym environment.)

---

6) Computer Vision — YOLO (Object Detection)

Steps (workflow)

1. Prepare images and bounding-box annotations (class, x,y,width,height).
2. Convert annotations to required format (e.g., COCO, YOLO txt).
3. Train model that predicts bounding boxes + class probabilities in a single pass.
4. During inference, model proposes many boxes; apply **non-max suppression (NMS)** to remove overlaps.
5. Output final bounding boxes and class labels with confidence scores.

Conceptual Python (usage example — prebuilt model)

```
# Example using a pretrained detect model (conceptual)
# Many implementations: ultralytics/yolov5, yolov8, etc.
import cv2
# model = load_yolo_model('yolov5s')  # conceptual placeholder
# results = model.detect(image)
# for box, cls, conf in results: draw box
```

(Training YOLO requires annotation conversion and GPU; for inference many repos provide ready-to-use APIs.)

---

7) Multi-Modal — CLIP (Contrastive Language–Image Pretraining)

Steps (workflow)

1. Collect large dataset of (image, caption) pairs.
2. Pass images through an image encoder and captions through a text encoder.
3. Train with a **contrastive loss** so matching image/text embeddings are close.
4. At inference, encode images and text; compute cosine similarity to rank matches.
5. Use for zero-shot image classification, image search, or multimodal retrieval.

Conceptual Python (Hugging Face CLIP)

```
from transformers import CLIPProcessor, CLIPModel
from PIL import Image

model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

image = Image.open("cat.jpg")
inputs = processor(text=["a photo of a cat", "a photo of a dog"], images=image, return_tensors="pt",
padding=True)
outputs = model(**inputs)
image_emb = outputs.image_embeds  # (1, dim)
text_embs = outputs.text_embeds   # (2, dim)
```

```
# similarity = (text_embs @ image_emb.T).softmax(dim=0)
```

---

8) Specialized — AlphaFold (protein structure prediction)

Steps (workflow, high level)

1. Input: primary amino-acid sequence.
2. Search sequence databases to build Multiple Sequence Alignments (MSAs) and identify templates — this provides evolutionary information.
3. Feed sequence + MSA + templates to structure prediction network (deep model trained on known structures).
4. Model outputs 3D coordinates of backbone and side chains, plus confidence scores (e.g., pLDDT).
5. Optionally refine and validate predicted structure (energy minimization, clash checks).

Conceptual Python (very high-level pseudocode)

```
sequence = "MKT..."  # amino-acid sequence
# Real AlphaFold requires databases and special runtime; pseudocode:
msa = run_msa_search(sequence)       # heavy step: search UniProt/etc.
templates = find_templates(sequence)  # optional
structure, confidence = alphafold_predict(sequence, msa, templates)
# save structure as PDB for downstream analysis
```

Note: AlphaFold's public code is complex, needs large reference databases and GPUs/TPUs. The above is schematic.

If you want, I can:

- Expand any single snippet into a runnable notebook (I can produce full training scripts for local use).
- Provide a short cheat-sheet of *key hyperparameters to tune* for each model.
- Show a visual diagram for one of the pipelines (e.g., BERT or GAN).

Which model would you like a runnable example for first?